

EmoMV: Cross-Modal Emotion-Conditioned Music Generation

Methods, Architecture, Training, and Experimental History

EmoMV Project

July 7, 2026

Contents

1	Introduction	2
2	Dataset and Preprocessing	3
2.1	Raw data	3
2.2	Manifest construction (Phase 1)	3
2.3	Train/val/test split (Phase 2)	4
2.4	Video decoding and frame sampling (Phase 3)	4
2.5	Frozen encoder feature extraction (Phase 4)	5
2.6	SlowFast baseline features	5
2.7	Text mood classifier (external asset)	5
2.8	512-d task-supervised video embedding caching	6
2.9	Audio target extraction for Stage C	6
3	Model Architectures	6
3.1	Stage A: baseline classifier (<code>SlowFastBaselineMLP</code>)	6
3.2	Stage A: video fusion classifier (<code>FusionClassifier</code>)	6
3.3	Stage B: cross-modal alignment (<code>AlignmentModel</code>)	7
3.3.1	Which embedding is projected, and why	7
3.4	Stage C: bridge network (<code>EmbeddingToConditioningBridge</code>)	7
3.5	MusicGen (frozen generative backbone)	8
3.6	Dual-conditioning module (<code>project/models/conditioning.py</code>)	8
4	Training Methodology	8
4.1	Stage A: classifier training	8
4.2	Stage B: label-anchored contrastive alignment training	9
4.3	Stage C, Milestone 1: single-source bridge training	10
4.4	Stage C extension: dual conditioning	10
4.5	Notebook-driven training	11
5	Evaluation Methodology	11
5.1	Stage A: classifier evaluation	11
5.2	Stage B: alignment evaluation	11
5.3	Stage C: qualitative generation evaluation	12

6	Results	13
6.1	Stage A: mood classification	13
6.2	Stage B: cross-modal alignment	14
6.3	Stage C: qualitative outcome	14
7	Full Experiment Log	14
8	Limitations and Discussion	16
9	Conclusion	17

1 Introduction

This report documents the full methodology of the EmoMV project: a three-stage pipeline that takes a video clip (or a piece of text) expressing one of five moods — *exciting*, *fear*, *tense*, *sad*, *relax* — and uses it to condition a pretrained music-generation model toward that mood, without ever fine-tuning the generator’s own weights. The pipeline is organized into three stages:

- **Stage A — unimodal mood classifiers.** A frozen-backbone video classifier (VideoMAE + CLIP fusion) and a fine-tuned text classifier (DistilRoBERTa) are each trained/used to predict one of 5 mood classes from a single modality.
- **Stage B — cross-modal alignment.** Small projection adapters map each classifier’s penultimate representation into a single shared, L2-normalized embedding space, trained with a label-anchored contrastive loss (no instance-level video–text pairs exist, only shared categorical labels).
- **Stage C — music generation conditioning.** A small “bridge” network maps a point in the shared space into MusicGen’s text-conditioning slot, so the mood vector can steer a frozen, pretrained MusicGen model at inference time. A later extension (*dual conditioning*) concatenates this mood signal with MusicGen’s own frozen T5 encoding of raw text, so genre/instrumentation detail is not discarded.

Every component that touches a pretrained model (VideoMAE, CLIP, DistilRoBERTa’s backbone in Stage B/C, MusicGen’s T5 encoder / EnCodec tokenizer / LM decoder) is kept fully frozen once it reaches Stage B or Stage C; only small adapters and the bridge network are ever trained on top. Figure 1 shows the full pipeline; frozen components are shaded blue, trained components orange, and components that were designed and partially executed but not concluded are shown dashed.

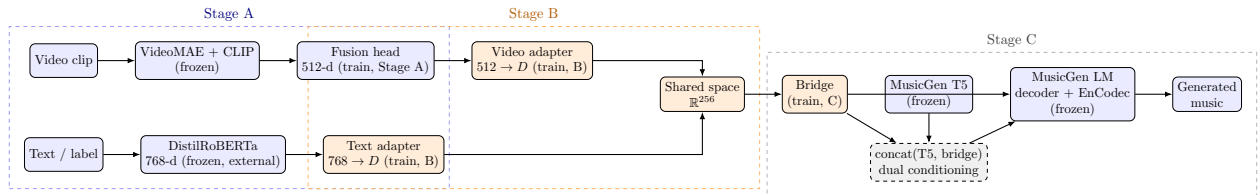


Figure 1: Full pipeline. Blue = frozen pretrained component. Orange = trained in this project. The dual-conditioning concatenation path (Section 4.4) is an extension of the original single-source bridge design.

2 Dataset and Preprocessing

2.1 Raw data

The project builds on the EmoMV dataset, distributed as three sub-collections (DS1, DS2, DS3), each with a headerless 7-column annotation CSV per split giving a clip’s relative directory, file stem, a `match_lbl` flag, and an emotion label. `match_lbl` distinguishes *MATCH* rows (video content and the labeled emotion agree by the dataset’s own construction) from *MISMATCH* rows (deliberately paired with a different, non-corresponding emotion, used in the original EmoMV benchmark for retrieval-style tasks). An exploratory data analysis (`emomv_eda.ipynb`, `eda_outputs/eda_summary.json`) preceded pipeline construction and reported: 6,452 physical `.mp4` files on disk, 0 corrupt/unreadable files, 0 annotation entries missing on disk, 3,178 *MATCH* pairs and 2,808 *MISMATCH* pairs, dominant resolution 640×360 (16:9, landscape), dominant FPS 25.0, dominant codec h264, and a median file size of 2.20 MB (15.89 GB on disk in total). The EDA also ran pairwise two-sample Kolmogorov–Smirnov tests on clip-duration distributions across DS1/DS2/DS3 and found statistically significant differences among all three sub-datasets — i.e., a real domain shift, not just a labeling artifact — which is the reason every downstream evaluation in this project is reported broken out per dataset source rather than only pooled (Section 5).

2.2 Manifest construction (Phase 1)

`project/preprocessing/build_manifest.py` is the single source of truth every later phase reads from. Its pipeline, in order:

1. **Parse annotations, MATCH-only.** Only rows with `match_lbl == 1` are kept (config key `match_lbl_keep: 1`) — *MISMATCH* rows are dropped entirely to avoid label ambiguity/leakage in a mood-classification setting.
2. **Emotion normalization.** Raw emotion strings are lower-cased and passed through an alias table (`emotion_aliases` in `config.yaml`: `happy→exciting`, `excitement→exciting`, `contentment→relax`, `relaxing→relax`, `fearful→fear`, `tension→tense`) before being mapped to one of 5 integer class ids (`exciting=0`, `fear=1`, `tense=2`, `sad=3`, `relax=4`).
3. **Drop missing files.** Any annotation row whose referenced `.mp4` is absent on disk is dropped.
4. **Container probing.** Every remaining file is opened with OpenCV to confirm it is decodable and to record FPS, frame count, and duration; unreadable clips are dropped.
5. **Exact deduplication.** Every clip is SHA-256 hashed; clips with identical hashes are exact duplicates and only the first-seen copy is kept. This step found **34 duplicate pairs**, of which **2 pairs had conflicting emotion labels** (the same physical video annotated with two different moods across sub-datasets) — these conflicts are resolved by keeping the first-seen annotation, and are logged to `project/data/duplicates_report.csv` for manual inspection rather than silently discarded.
6. **Group-id derivation.** A `source_group_id` is derived per clip so that segments of the same underlying source video are never split across train/val/test (Section 2.3): DS1 stems of the form `..name-2-of-8.mp4` have the `-k-of-n` suffix stripped via regex; DS3 stems of the form `<youtube_id>.02.mp4` have the trailing 2-digit segment index stripped; DS2 stems are used as-is (bare YouTube ids, no observed segmentation).

7. Duration-outlier windowing. Clips whose duration exceeds `max_duration_s + tolerance_s = 30 + 5 = 35s` are treated as genuine outliers (not the much larger set of clips that merely round to 30.0–30.5s from encoding) — **20 such outliers** were found (mean duration 87.9s, max 392.4s), logged to `project/data/duration_outliers.csv`, and split into non-overlapping `window_s = 30s` windows (`project/preprocessing/duration_windows.py`); a trailing remainder shorter than `min_window_remainder_s = 5s` is dropped rather than kept as an undersized window. No video is re-encoded — only `(start_frame, end_frame)` ranges are computed; the dataset class slices lazily at decode time.

Resulting manifest. The final manifest (`project/data/manifest.csv`) contains **3,189 rows** (windowed sub-clips included) covering **3,144 unique parent videos**. Per-dataset-source counts: DS1 = 2,653, DS2 = 308, DS3 = 228. Per-emotion counts: exciting = 689, fear = 537, tense = 459, sad = 594, relax = 910 (a roughly 2:1 imbalance between the largest and smallest class).

2.3 Train/val/test split (Phase 2)

`project/preprocessing/split.py` rebuilds the train/val/test assignment from scratch rather than using EmoMV’s own official split, because the official split was found to leak: clips segmented from the same source video (the *-k-of-n* DS1 naming convention) land in different splits under it — 37 groups span train/val, 23 span train/test, and 34 span val/test in the official assignment. The replacement algorithm is a **greedy iterative stratification**: `source_group_id` groups are shuffled deterministically (seed 42), then each group is assigned, one at a time, to whichever split currently has the largest *deficit* against its target fraction for that group’s (majority) emotion class. This gives precise 70/15/15 control (`train_frac = 0.70`, `val_frac = 0.15`, `test_frac = 0.15`) while guaranteeing zero cross-split group leakage by construction — verified post-hoc by an assertion that no `source_group_id` has more than one unique split value. The resulting split sizes, broken out per dataset source, are:

Split	Rows	DS1	DS2	DS3
train	2,232	1,870	190	172
val	479	395	59	25
test	478	388	59	31

DS1 dominates at 83% of all clips, a fact worth keeping in mind when reading the per-dataset results in Section 6. Class-imbalance weighting (Section 4.1) uses inverse-frequency weights derived from the training-set class counts; the EDA’s own precomputed reference values were exciting 0.858, fear 1.104, tense 1.321, sad 1.084, relax 0.800 (imbalance ratio $\approx 1.65\times$ between the most- and least-frequent class).

2.4 Video decoding and frame sampling (Phase 3)

`project/datasets/video_dataset.py`’s `EmoMVVideoDataset` turns a manifest row into a (T, C, H, W) tensor of frames in $[0, 1]$ (per-encoder normalization is applied later, in `models/encoders.py`, so the same decoded clip can feed both frozen encoders without re-decoding). $T = 16$ frames (`num_frames`) are sampled by dividing the clip’s `[start_frame, end_frame)` range into 16 equal segments; in `mode="train"` a uniformly random position within each segment is sampled (temporal jitter), while in `mode="val"/"test"` the deterministic segment midpoint is used,

so validation/test embeddings are reproducible across runs. Frames are then resized so the shorter edge is 256 px and center-cropped to 224×224 (`resize_shorter_edge, crop_size`), then converted BGR→RGB. Decode failures are handled per-frame (a failed frame is zero-padded, logged via a dedicated logger to `project/logs/dataset_errors.log`) rather than failing the whole batch. Feature extraction (Phase 4) always uses `mode="val"` deterministic sampling, even for the training split, since embeddings are computed once and cached rather than resampled every epoch.

2.5 Frozen encoder feature extraction (Phase 4)

`project/features/extract_features.py` runs two frozen HuggingFace encoders over every manifest row and caches one tensor per clip per encoder (skipping any clip whose cache file already exists, so re-running is idempotent):

- **VideoMAE** (MCG-NJU/`videomae-base`): frames are normalized with VideoMAE’s own per-channel mean/std, passed through the frozen model, and the `last_hidden_state` is mean-pooled over the token/time dimension to a **768-d** vector.
- **CLIP** (openai/`clip-vit-base-patch32`): each of the 16 frames is individually normalized with CLIP’s own mean/std, passed through `get_image_features`, and the resulting per-frame 512-d embeddings are mean-pooled over the 16 frames to a **512-d** vector.

Both encoders are wrapped with `requires_grad=False` on every parameter and run under `torch.no_grad()` at all times; extraction uses batch size 8 (`encoders.batch_size`) with device preference `["mps", "cuda", "cpu"]`.

2.6 SlowFast baseline features

As a reference point that requires no new video decoding, `project/features/extract_slowfast_baseline.py` re-aligns the dataset’s own shipped, pre-extracted SlowFast features (`.h5` files, one row per line of the *original* MATCH+MISMATCH annotation CSV, in the same row order — verified directly by comparing row counts) with the MATCH-only manifest, keeping only `match_lbl==1` rows and caching each as a `features/slowfast/<parent_video_id>.pt` (2,304-d). SlowFast features exist only per whole original clip, not per window, so a windowed manifest row is keyed by its `parent_video_id` rather than its own `video_id` when loaded (`CachedEmbeddingDataset`’s `key_column` argument handles this distinction generically).

2.7 Text mood classifier (external asset)

The text-side mood classifier (`project/text_mood_clf_encoder/text_mood_clf`) is a **pretrained, externally fine-tuned** `distilroberta-base` (`RobertaForSequenceClassification`; 6 transformer layers, hidden size 768, 12 attention heads, vocab 50,265) reporting test macro-F1 ≈ 0.80 on the same 5-class label set (labels: `excited`, `fear`, `tense`, `sad`, `relax` — identical ordering to the video side except for the “exciting” vs. “excited” lexical difference, which is a deliberate, useful coincidence that avoids needing a label-id remapping table between the two modalities). Unlike the video fusion head, this is a genuine end-to-end fine-tune: RoBERTa’s own backbone weights were updated during its (external) training, not just a linear head on top of frozen features. This project consumes it purely as a frozen encoder (`FrozenTextMoodEncoder` in `project/models/encoders.py`), exposing its **768-d penultimate representation** (post `dense+tanh`, pre `out_proj`) rather than its final logits, by manually replaying `roberta` →

`classifier.dropout` $\rightarrow$ `classifier.dense` $\rightarrow$ `tanh` $\rightarrow$ `classifier.dropout`. This reconstruction was verified to reproduce the model’s own `.logits` output exactly before being relied upon, and is symmetric to how `FusionClassifier`’s own penultimate 512-d layer is used on the video side (Section 3.3.1).

2.8 512-d task-supervised video embedding caching

Once the Stage A fusion classifier is trained (Section 3.2), `project/features/extract_video_embedding_512.py` caches its 512-d *penultimate* activation (everything in `FusionClassifier` except the final `Linear(512,5)`) for every clip to `features/video_embedding_512/<video_id>.pt`. This — not the raw 1280-d VideoMAE+CLIP concatenation, and not the 5-class logits — is the representation Stage B’s video adapter projects from (Section 3.3.1).

2.9 Audio target extraction for Stage C

`project/features/extract_audio_targets.py` extracts each MATCH clip’s own audio track via `ffmpeg` (`-vn -ac {channels} -ar {sample_rate}`), respecting a windowed sub-clip’s (`start_frame`, `end_frame`) boundaries by converting them to `-ss/-t` seconds via the clip’s own FPS. Output is resampled to **32 kHz mono** — the input format MusicGen-small’s EnCodec tokenizer expects (`stage_c.audio_sample_rate=32000`, `audio_channels=1`). This is a deliberate reuse of an existing resource: since EmoMV’s MATCH clips have video emotion equal to music emotion *by the dataset’s own construction*, every one of these clips’ own soundtracks is already a mood-consistent generation target, requiring no new data collection — only extraction. Two caveats are documented but *not* acted upon in this project (Section 7): the extracted audio is a music-video soundtrack that may contain vocals, sound effects, or dialogue rather than a clean instrumental bed, and it is used as-is.

3 Model Architectures

3.1 Stage A: baseline classifier (SlowFastBaselineMLP)

A simple reference classifier over the shipped, pre-extracted SlowFast features, established before investing in the frozen VideoMAE+CLIP fusion model:

$$\mathbb{R}^{2304} \xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{ReLU, Dropout}(0.3)} \mathbb{R}^{256} \xrightarrow{\text{Linear}} \mathbb{R}^5.$$

3.2 Stage A: video fusion classifier (FusionClassifier)

Concatenates the frozen VideoMAE (768-d) and CLIP (512-d) embeddings (Section 2) into a 1280-d vector and classifies it with a single hidden layer:

$$\mathbb{R}^{1280} \xrightarrow{\text{Linear}} \mathbb{R}^{512} \xrightarrow{\text{GELU, Dropout}(0.3)} \mathbb{R}^{512} \xrightarrow{\text{Linear}} \mathbb{R}^5.$$

The forward pass supports a `return_embedding=True` flag that returns the 512-d activation *after* GELU+Dropout but *before* the final `Linear(512,5)` — indexing directly into the underlying `nn.Sequential` rather than restructuring the module, so an already trained checkpoint’s `state_dict` layout is unaffected. Only VideoMAE and CLIP’s own weights are frozen here; this fusion head itself is trained from scratch (Section 4.1).

3.3 Stage B: cross-modal alignment (AlignmentModel)

Two structurally symmetric MLP adapters project each modality’s frozen, task-supervised penultimate embedding into a shared $D = 256$ -dimensional space:

$$\text{video_adapter} : \mathbb{R}^{512} \rightarrow \mathbb{R}^{256}, \quad \text{text_adapter} : \mathbb{R}^{768} \rightarrow \mathbb{R}^{256},$$

each implemented identically apart from input width as

$$\text{Linear}(d_{\text{in}}, 2D) \rightarrow \text{LayerNorm} \rightarrow \text{GELU} \rightarrow \text{Dropout}(0.15) \rightarrow \text{Linear}(2D, D),$$

followed by L2 normalization $\hat{z} = z/\|z\|_2$ so downstream similarities are pure cosine similarities (the same convention CLIP uses). A learnable temperature (`LogitScale`) is parameterized in log-space as CLIP does, $\text{logit_scale} = \exp(\min(\log s, \log s_{\text{max}}))$, initialized at $s = 1/0.07 \approx 14.2857$ and clamped to a maximum of 100. `AlignmentModel` bundles both adapters and the temperature into a single checkpointable module with `encode_video/encode_text` methods.

3.3.1 Which embedding is projected, and why

Stage B deliberately does *not* project the raw 1280-d VideoMAE+CLIP concatenation or RoBERTa’s raw token embeddings. It projects the *penultimate, task-supervised* representation from each existing Stage A classifier (Section 3.2, Section 2.7). Both are already the result of supervised training toward the same 5-class taxonomy, so the adapters only need to learn a geometric alignment of two already-discriminative spaces, rather than starting from generic, task-agnostic backbone features.

3.4 Stage C: bridge network (EmbeddingToConditioningBridge)

MusicGen does not consume a single conditioning vector; its cross-attention expects a *sequence* of $(T, 768)$ hidden states (T5’s hidden size), the same shape its own frozen T5 text encoder would normally produce for a text prompt. The bridge’s job is to turn Stage B’s single 256-d point z into such a sequence:

1. **Input projection.** $z \in \mathbb{R}^{256}$ is linearly projected to $\mathbb{R}^{512}$ (`hidden_dim`) and treated as a single “memory” token, $(B, 1, 512)$.
2. **Learned queries.** A learned parameter tensor of shape $(8, 512)$ (`seq_len=8`, initialized $\mathcal{N}(0, 0.02^2)$) is broadcast across the batch as the target sequence for a standard `nn.TransformerDecoder` (2 layers, 4 heads, feed-forward dim = $2 \times 512 = 1024$, dropout 0.1, batch-first). Each decoder layer lets the 8 queries self-attend among themselves and cross-attend to the single memory token.
3. **Output projection.** The decoded $(B, 8, 512)$ sequence is linearly projected to $\mathbb{R}^{768}$ (MusicGen’s T5 hidden size) and passed through a final `LayerNorm`.

The bridge always emits a fixed-length, unpadded 8-token sequence, so its accompanying attention mask is simply all-ones. Only this bridge network is ever trained in Stage C (Section 4.3); everything upstream (Stage A/B) and downstream (MusicGen) is frozen.

3.5 MusicGen (frozen generative backbone)

The generator is `facebook/musicgen-small` via `transformers.MusicgenForConditionalGeneration`: an autoregressive transformer over discrete EnCodec audio tokens, text-conditioned via a T5 encoder whose hidden-state sequence is cross-attended by every decoder layer. All of MusicGen’s parameters — the T5 text encoder, the EnCodec tokenizer/decoder, and the LM decoder itself — are frozen throughout this project (`requires_grad=False` on every parameter, model kept in `.eval()`). Verified directly against the installed `transformers` source (`modeling_musicgen.py`): if `encoder_outputs` is not supplied to `.forward()/.generate()`, MusicGen calls its own T5 encoder on `input_ids`; if it *is* supplied (as a tuple), it is wrapped directly into a `BaseModelOutput` and used as-is, regardless of its source or sequence length — this is the hook both the single-source bridge and the dual-conditioning extension exploit. A config-loading quirk in the installed `transformers` version is fixed defensively in both training scripts: `musicgen.config.decoder.decoder_start_token_id` loads as `None` even though `generation_config.decoder_start_token_id` (the reserved EnCodec pad token, 2048) is set correctly, and is copied over before training since `shift_tokens_right` needs it.

3.6 Dual-conditioning module (`project/models/conditioning.py`)

A small set of stateless helpers supporting the concatenated-conditioning extension (Section 4.4):

- `build_musicgen_tokenizer`: loads the T5 tokenizer shipped with the MusicGen checkpoint (the same tokenizer stock MusicGen uses internally).
- `t5_encode`: runs raw text through MusicGen’s own frozen `musicgen.text_encoder` under `no_grad`, returning *both* the hidden-state sequence ($B, T_{\text{text}}, 768$) and its padding mask — exactly what stock MusicGen computes internally for a text prompt, just exposed for reuse.
- `concat_conditioning`: concatenates the T5 sequence and the bridge’s sequence along the time axis, and concatenates their masks correspondingly:

$$\text{cond} = [T5(\text{text}); \text{bridge}(z)] \in \mathbb{R}^{B \times (T_{\text{text}} + T_{\text{bridge}}) \times 768}, \quad \text{mask} = [\text{mask}_{T5}; \mathbf{1}_{T_{\text{bridge}}}]$$

- `label_template_text`: builds the raw-text input for a clip from its integer emotion label via a fixed template (e.g. "`{emotion}` instrumental music" + label 4 $\rightarrow$ "relax instrumental music"), since EmoMV clips carry no per-clip captions.

4 Training Methodology

4.1 Stage A: classifier training

Both the SlowFast-baseline MLP and the VideoMAE+CLIP fusion classifier share one training loop (`project/training/loop.py`, `train_classifier`):

- **Optimizer:** `torch.optim.Adam`, learning rate 3×10^{-4} , weight decay 10^{-4} (`training.lr`, `training.weight_decay`).
- **Batching:** batch size 32, 4 dataloader workers.
- **Class imbalance:** `class_imbalance_strategy` supports two interchangeable options, selected via config (Section 7): "`class_weights`" (inverse-frequency weights $w_k = N/(K \cdot n_k)$)

fed into a weighted `nn.CrossEntropyLoss`) — the option actually used for all reported runs — or `"weighted_sampler"` (a `WeightedRandomSampler` built from the same per-class weights).

- **Early stopping:** patience 10 epochs, monitored on validation macro-F1 (maximize), up to 100 epochs.
- **Checkpointing:** the best-macro-F1 epoch’s model/optimizer state is saved to `fusion_classifier_best.pt / baseline_slowfast_mlp_best.pt`.

The fusion classifier uses `fusion_hidden_dim=512`, `fusion_dropout=0.3`; the baseline uses `baseline_hidden_dim=256`, `baseline_dropout=0.3`. In the recorded run, the SlowFast baseline early-stopped at **epoch 44** (best validation macro-F1 0.525) while the fusion classifier converged far faster, early-stopping by **epoch 10** (best validation macro-F1 0.807) — frozen VideoMAE/CLIP embeddings evidently carry substantially more emotion-discriminative signal than the shipped SlowFast features even into an identically-sized MLP head. Validation and test macro-F1 for the fusion model are close (0.807 vs. 0.794), giving no strong indication of overfitting to the validation split.

4.2 Stage B: label-anchored contrastive alignment training

`project/training/train_alignment.py` trains only the two Stage B adapters and the temperature (Section 3.3); the frozen encoders that produced the cached embeddings are never touched.

The pairing problem. Standard CLIP-style contrastive training assumes naturally occurring (image, caption) instance pairs. This project has no such pairs — only ~3,144 videos and a text corpus, each independently carrying one of the same 5 categorical mood labels, with no instance-level video↔sentence correspondence. Rather than collect or synthesize pairs, Stage B uses exactly **5 fixed text anchors** — the 5 class-name strings themselves (`load_text_anchors` in `project/training/alignment_common.py`), each encoded once through the frozen text encoder into a (5,768) tensor. This is documented in the code as a deliberate, honest simplification (no larger text training corpus was available), not an oversight.

Loss (`project/training/losses.py`). With the text side reduced to 5 fixed anchors, the video→text direction is a plain 5-way softmax cross-entropy (mathematically InfoNCE with the 4 other anchors as in-batch negatives):

$$\mathcal{L}_{v2t} = -\frac{1}{N} \sum_i \log \frac{\exp(s_{i,y_i})}{\sum_{c=1}^5 \exp(s_{ic})}, \quad s_{ic} = \text{logit_scale} \cdot \hat{v}_i \cdot \hat{t}_c.$$

The text→video direction is the multi-positive supervised-contrastive generalization (Khosla et al., 2020), since many videos in a batch can share a given anchor’s class:

$$\mathcal{L}_i = -\frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(s_{ip})}{\sum_a \exp(s_{ia})}, \quad P(i) = \{\text{videos in-batch sharing anchor } i\text{'s class}\}.$$

The total loss is the mean of the two directions, $\mathcal{L} = \frac{1}{2}(\mathcal{L}_{v2t} + \mathcal{L}_{t2v})$. No auxiliary classification loss term is present in the implemented loss function (one was proposed in the planning document as a safeguard against representation collapse, but the shipped `label_anchored_contrastive_loss` implements only the contrastive terms above — Section 7).

Optimization. AdamW, learning rate 10^{-3} , weight decay 10^{-4} , batch size 128, up to 200 epochs, early stopping patience 15 epochs on validation video→text accuracy (maximize). `shared_dim=256`, `adapter_dropout=0.15`. Because there are exactly 5 fixed text anchors, video→text Recall@1 collapses to simple 5-way classification accuracy against the anchors as class prototypes — this identity is used directly as the monitored/early-stopping metric. The recorded run early-stopped at **epoch 25**, with a best *validation* video→text accuracy of **0.8205**; the *held-out test* accuracy reported in Section 6 (0.8096) is evaluated separately, after training, on the test split the early-stopping criterion never saw.

4.3 Stage C, Milestone 1: single-source bridge training

`project/training/train_bridge.py` pairs each clip’s cached 512-d video embedding with its own extracted audio waveform (`VideoEmbeddingAudioDataset`) — no additional pairing effort is needed, since a MATCH clip’s video and audio are already the same underlying file. Waveforms are truncated/zero-padded to `clip_duration_s=10s` at 32 kHz.

Objective. MusicGen’s own teacher-forced next-token cross-entropy loss over EnCodec audio codes — the same objective MusicGen’s text-conditioning pathway was originally trained with; only the conditioning *source* changes. The target waveform is encoded to EnCodec codes via `musicgen.audio.encoder.encode` (asserting exactly one EnCodec frame, i.e. chunking disabled) and reshaped to the `(B, seq_len, num_codebooks)` layout `MusicGenForConditionalGeneration.forward` expects for its `labels` argument (distinct from the `(B*num_codebooks, seq_len)` layout used for `input_ids`). Each step: `z = alignment_model.encode_video(video_emb)` (frozen, no grad) $\rightarrow$ `cond = bridge(z)` (only tensor with gradients) $\rightarrow$ `musicgen(encoder_outputs=(cond,), attention_mask=..., labels=codec_labels)`.

Optimization. AdamW on `bridge.parameters()` only, learning rate 3×10^{-4} , weight decay 10^{-5} , batch size 4, early stopping patience 5 (minimize validation loss). The *originally committed* configuration ran this as a deliberately small-scale sanity check — `subset_size=300` training clips (validation subset = $\max(300/5, 10) = 60$), `max_epochs=2` — consistent with the planning document’s recommendation to validate cheaply that the shared space carries usable conditioning signal at all before scaling up. The checkpoint from this run is `bridge_best.pt`.

4.4 Stage C extension: dual conditioning

A later, currently-uncommitted round of work (config changes and a new `project/models/conditioning.py` module, both present in the working tree but not yet merged into version control at the time of this report) extends Milestone 1 as proposed in `project/docs/text_conditioning_bridge_plan.pdf`: rather than the bridge output *replacing* MusicGen’s T5 conditioning outright, MusicGen’s own frozen T5 encoder is run on raw text (a per-clip templated string, Section 3.6, since no real per-clip captions exist) and its output is **concatenated** with the bridge’s mood-steering sequence before cross-attention (Section 3.6). The motivation: the original design gives MusicGen *only* the collapsed 5-class mood point as its entire conditioning input, discarding any richer descriptive content (instrumentation, genre, tempo) that raw text could carry.

What changed mechanically. `train_bridge.py`'s dataset and training loop gained a `dual_conditioning` branch (config flag `stage_c.dual_conditioning: true`): each batch's raw text is built from its integer label via `label_template_text` with template `"{emotion} instrumental music"` (`text_template`, max length 64 tokens), encoded through the frozen T5 (`t5_encode`, no gradient), and concatenated with the bridge's own output (`concat_conditioning`) before being passed to MusicGen. **Only the bridge still receives gradients** — T5 is run under `no_grad` either way, so this adds no optimizer state relative to Milestone 1. Because the old `bridge_best.pt` checkpoint was trained assuming its output was the *sole* conditioning source, it is not reused as-is: a separate checkpoint name, `bridge_dual_best.pt` (`bridge_checkpoint_name` config key), keeps the two trainings independent so the original bridge-only baseline remains available for comparison. A `resume: true` config flag was also added, allowing training to reload an existing checkpoint (model, optimizer state, epoch counter, and best validation score) and continue rather than restart from scratch. Alongside this, `max_epochs` was raised from 2 to 200 and `subset_size` from 300 to 9,999 (i.e. effectively uncapped, using the entire available train/val pool with cached audio) for this extended run. A `bridge_dual_best.pt` checkpoint exists in `project/training/checkpoints/`, confirming this training was actually executed (not merely planned).

4.5 Notebook-driven training

Every training script above has a parallel, cell-by-cell notebook counterpart intended for interactive runs with live progress bars and inline audio playback rather than being invoked as a background process: `05_train.ipynb` (Stage A), `07_alignment.ipynb` (Stage B), and `08_bridge.ipynb` (Stage C, updated in place to add the dual-conditioning path and its three-way generation/ablation cells, Section 5.3).

5 Evaluation Methodology

5.1 Stage A: classifier evaluation

`project/evaluation/evaluate.py` loads a trained checkpoint (baseline or fusion), predicts over the held-out test split, and reports accuracy, macro-F1, and weighted-F1 both **overall** and broken out **per dataset source (DS1/DS2/DS3)** — the per-dataset breakdown exists specifically because the EDA's confirmed domain shift (Section 2) means a model that only fits DS1 should show up as a DS2/DS3 gap in this report, not just a mediocre pooled number. Confusion matrices are rendered and saved per breakdown (`*_confusion_matrix_{overall,DS1,DS2,DS3}.png`) alongside a JSON metrics dump.

5.2 Stage B: alignment evaluation

`project/evaluation/evaluate_alignment.py` reports, again overall and per dataset source:

- **Video→text accuracy / Recall@1:** with only 5 fixed anchors, this is exactly 5-way classification accuracy using the anchors as class prototypes (nearest anchor by cosine similarity). Text→video retrieval is explicitly *not* reported — with no diversity on the text side, there is nothing meaningful to retrieve *among* on that side, and the code comments document this omission rather than silently reporting only the flattering half.
- **Per-class recall.**

- **Modality-gap diagnostic:** for each class, the cosine similarity between that class’s video-embedding centroid and its text anchor — CLIP-family models are documented to retain a persistent “modality gap” (the two modalities occupying separate sub-manifolds even after contrastive training), so this is measured explicitly rather than assumed away.
- **Qualitative t-SNE plot** of the shared space (video points colored/shaped by class, text anchors shown as large stars), to visually check that same-label points from both modalities cluster together rather than forming two same-colored but separate modality islands.

5.3 Stage C: qualitative generation evaluation

Stage C has **no automatic acoustic metric** (e.g. no Fréchet Audio Distance or CLAP score is computed anywhere in the codebase) — evaluation is a deliberate, documented *manual-listening* protocol via `project/evaluation/generate_samples.py`, chosen because the teacher-forced LM loss used for training cannot by itself distinguish “the decoder generates coherent audio” from “the decoder generates audio that reflects the requested mood.” Samples are generated from two sources, on purpose:

1. **Text anchors** (the 5 fixed class-name strings) — out-of-distribution for the bridge, since it only ever trained on video embeddings; tests whether Stage B’s alignment actually transfers to steering MusicGen from text alone.
2. **Real video embeddings** from held-out test clips (one per emotion class) — in-distribution, what the bridge was actually trained on.

`guidance_scale=1.0` is set explicitly in every generation call: MusicGen’s default classifier-free-guidance path duplicates the batch against an internal null/unconditional embedding when conditioning via raw `input_ids`, which does not apply when custom `encoder_outputs` are supplied directly — disabling it avoids a shape mismatch rather than silently producing incorrect output.

Three-way ablation (dual-conditioning extension). Once dual conditioning was implemented, sample generation was extended to produce, for every text-anchor and video-embedding example, three variants:

- **t5only:** `cond = T5(text)` alone — the stock-MusicGen baseline the combined approach must beat on mood-consistency, since it is free (no training at all).
- **bridgeonly:** `cond = bridge(z)` alone — the original Milestone 1 design, the baseline the combined approach must not regress against.
- **combined:** `cond = concat(T5(text), bridge(z))` — the proposed approach.

This produced **40 output .wav files** in `project/evaluation/results/generated_samples/`: the original 10 single-source samples (5 text-anchor + 5 video-embedding, one per class, from the Milestone-1-only `bridge_best.pt` run) plus 30 further samples from the three-way ablation (5 classes $\times$ 2 sources $\times$ 3 conditioning variants, using the retrained `bridge_dual_best.pt`). An **attention diagnostic** — inspecting the decoder’s `cross attentions` output to check whether it collapses onto a single branch of the concatenated sequence — was recommended in the planning document as a cheap additional check but was **not implemented in code**; only manual listening was actually carried out.

6 Results

6.1 Stage A: mood classification

Model	Accuracy	Macro-F1	Weighted-F1
SlowFast baseline MLP (overall, $n = 478$)	50.63%	0.479	0.503
DS1 ($n = 388$)	54.90%	0.504	0.543
DS2 ($n = 59$)	33.90%	0.338	0.380
DS3 ($n = 31$)	29.03%	0.259	0.319
VideoMAE+CLIP fusion (overall, $n = 478$)	80.13%	0.794	0.803
DS1 ($n = 388$)	86.60%	0.860	0.867
DS2 ($n = 59$)	52.54%	0.473	0.545
DS3 ($n = 31$)	51.61%	0.366	0.550

The fusion model improves substantially over the SlowFast baseline on every split, but the DS1-vs-DS2/DS3 gap present in the baseline persists (and is not closed) in the fusion model, consistent with the EDA’s confirmed cross-dataset domain shift (Section 2) — DS2 and DS3 together contribute only 287 of the 3,189 manifest rows (9.0%), so both models see substantially less DS2/DS3 training signal than DS1.

Per-class breakdown (fusion model, overall test set).

Class	Precision	Recall	F1	Support
exciting	0.896	0.827	0.860	104
fear	0.655	0.889	0.754	81
tense	0.783	0.691	0.734	68
sad	0.766	0.809	0.787	89
relax	0.898	0.779	0.835	136

tense is the weakest class by F1; **exciting/relax** are strongest. The confusion matrix shows errors clustering between *adjacent* regions of the valence-arousal plane (**tense/fear** are both negative-valence; **relax/sad** are both low-arousal) and essentially never crossing to opposite quadrants (**exciting** is almost never confused with **sad**) — a positive signal that the fused embedding captures real affective structure rather than a spurious correlation. The single largest error mode is **tense**→**fear** (25.0% of true **tense** clips misclassified as **fear**).

Small-sample caveat on the DS2/DS3 breakdown. Part of the DS1-vs-DS2/DS3 gap above is a small-sample artifact, not purely a generalization failure: per-class test support is very uneven across datasets (e.g. DS3 has **zero** true **fear** examples and only 4 **tense** examples, all 4 of which are misclassified as **fear**). A macro-F1 averaged over such thin per-class support is high-variance, so the DS2/DS3 numbers should be read as “consistent with a real domain gap, but not statistically conclusive at this sample size,” not as a firm generalization-failure claim.

6.2 Stage B: cross-modal alignment

Split	Video→text accuracy	<i>n</i>
Overall	80.96%	478
DS1	88.66%	388
DS2	50.85%	59
DS3	41.94%	31

Per-class recall (overall): excited 0.808, fear 0.877, tense 0.706, sad 0.798, relax 0.831. Modality-gap cosine similarities (overall, own-class video centroid vs. text anchor): excited 0.709, fear 0.728, tense 0.694, sad 0.696, relax 0.722 — all comfortably positive and of similar magnitude across classes, indicating the adapters pull same-label video/text representations together to a broadly comparable degree per class, with no single class catastrophically misaligned. The same DS1-vs-DS2/DS3 gap seen in Stage A reappears here (88.7% vs. 50.9%/41.9%), unsurprising since Stage B’s video adapter is projecting from the Stage A fusion embedding directly and inherits its domain-shift sensitivity; DS3’s **fear** class in particular had zero test examples, hence the **null** recall/modality-gap entries recorded for that cell.

6.3 Stage C: qualitative outcome

No scalar metric is reported for Stage C by design (Section 5.3); the concrete, verifiable output of this stage is the existence of two independently trained bridge checkpoints (**bridge_best.pt**, **bridge_dual_best.pt**) and 40 generated **.wav** samples spanning both the original single-source design and the three-way t5only/bridgeonly/combined ablation, intended for manual listening assessment of mood-consistency and content specificity rather than an automatic scoreboard.

7 Full Experiment Log

This section enumerates every experiment/configuration option this project’s code supports or was built to support, regardless of whether it is the one actually used in the final reported numbers above — including alternatives that were implemented but not selected, and extensions that were designed but never executed.

Experiment / option	Status	Notes
SlowFast + MLP classifier	Implemented & run	Reference baseline; superseded by the fusion model but kept as a permanent comparison point in evaluate.py .
VideoMAE+CLIP fusion classifier	Implemented & run	Adopted Stage A video model; large accuracy jump over SlowFast baseline (Section 6).
Class-imbalance: class_weights	Implemented & run	The strategy actually selected in config.yaml for all reported Stage A runs.

Experiment / option	Status	Notes
Class-imbalance: <code>weighted_sampler</code>	Implemented, not adopted	Fully implemented as an alternate branch in <code>training/loop.py</code> (<code>WeightedRandomSampler</code> over inverse-frequency weights) but never the active config value for any run behind the reported results.
Stage B shared dimension $D = 256$	Implemented & run	Adopted; chosen to regularize against the modest (~ 3.2 k video / label-anchor text) data volume.
Stage B shared dimension $D = 512$	Proposed, not implemented	Explicitly proposed as a fallback “only if retrieval metrics plateau below what a linear probe on raw features already achieves”; that plateau condition was not observed (Section 6), so $D = 512$ was never trained.
Label-anchored supervised contrastive alignment (5 fixed text anchors)	Implemented & run	The only Stage B training strategy implemented; a generalization of Khosla et al. (2020) to a cross-modal, label-shared setting with no instance-level pairs.
Auxiliary classification loss on top of the contrastive objective	Proposed, not implemented	Recommended in <code>cross_modal_alignment_plan.pdf</code> as a safeguard against representation collapse with only 5 anchor classes ($\mathcal{L} = \mathcal{L}_{\text{contrastive}} + \lambda \mathcal{L}_{\text{CE}}$, $\lambda \approx 0.1\text{--}0.3$); the shipped <code>losses.py</code> implements only the contrastive terms, so this safeguard was never actually added to the training run.
Stage C Milestone 1 — bridge-only conditioning (<code>bridge_best.pt</code>)	Implemented & run	Small-subset sanity run (300 train clips, 2 epochs) per the plan’s “validate cheaply first” recommendation; confirmed the shared-space signal produces qualitatively mood-relevant audio before any larger investment.
Stage C dual conditioning — concatenated T5+bridge (<code>bridge_dual_best.pt</code>)	Implemented & run	Retrained on the full available train/val pool (subset_size raised to 9,999, i.e. effectively uncapped) for up to 200 epochs with checkpoint-resume support added; produced the three-way t5only/bridgeonly/combined ablation samples (Section 5.3). At the time of this report the supporting code (<code>config.yaml</code> edits, <code>project/models/conditioning.py</code>) is present in the working tree but not yet committed to version control.

Experiment / option	Status	Notes
Per-clip video captions for the dual-conditioning video-anchored regime	Proposed, not implemented	The plan documents this as strictly more informative than a template string if available; EmoMV videos have no per-clip captions, so a fixed template (" <code>{emotion}</code> <code>instrumental music</code> ") was used instead, and real captioning (e.g. via an off-the-shelf video-captioning model) was never pursued.
Cross-attention balance diagnostic for dual conditioning	Proposed, not implemented	Recommended as a cheap check for whether the decoder collapses onto one conditioning branch; no code reads out <code>cross attentions</code> anywhere in the repository, so this diagnostic was never run — only manual listening was performed.
Vocal/instrumental separation of Stage C audio targets	Proposed, not implemented	Documented as a caveat (EmoMV soundtracks may contain vocals/dialogue, not clean instrumental beds) but explicitly <i>not</i> acted on for Milestone 1 or the dual-conditioning extension; audio is used as extracted.
Stage C Milestone 2 — LoRA/adaptor fine-tuning of MusicGen’s cross-attention blocks	Proposed, not implemented	Proposed as the next step only if Milestone 1/dual-conditioning results underperform; explicitly flagged as resource-limited on the project’s Apple Silicon hardware, contingent on renting cloud GPU time. Not implemented.
Stage C Milestone 3 — full/partial MusicGen fine-tuning	Proposed, not implemented	Proposed as a stretch goal only if Milestone 2 plateaus <i>and</i> more paired data becomes available (e.g. supplementing EmoMV audio with a public mood-tagged corpus such as MTG-Jamendo). Not implemented.

8 Limitations and Discussion

- **Domain shift across sub-datasets.** DS2 and DS3 together are only 9.0% of the manifest (287/3,189 rows) and their clip-duration distributions differ significantly from DS1’s (confirmed by pairwise KS tests in the EDA). Every reported metric in this project — Stage A accuracy/F1 and Stage B retrieval accuracy alike — drops sharply on DS2/DS3 relative to DS1, indicating the pipeline’s learned components (fusion head, adapters) are fit substantially more to DS1’s distribution.
- **No instance-level text supervision for Stage B.** The text side of cross-modal alignment is exactly 5 fixed class-name strings, not a diverse text corpus; this buys coarse mood separation but not fine-grained semantic alignment within a class (two very different scenes sharing a label land at nearly the same point in the shared space by construction).

- **No automatic generation-quality metric.** Stage C’s evaluation is entirely manual listening; no Fréchet Audio Distance, CLAP score, or downstream mood-classifier-on-generated-audio metric is computed, so claims about the dual-conditioning extension’s benefit over the bridge-only baseline are qualitative, not statistically established.
- **Compute constraints.** All training in this project targets Apple Silicon (MPS) first; Stage C Milestones 2/3 (any fine-tuning that touches MusicGen’s own weights) are explicitly deferred pending rented GPU compute, since `audiocraft`/large-model gradient memory footprints are considered infeasible on the current local hardware.
- **Soundtrack audio quality.** Stage C’s generation targets are music-video soundtracks extracted as-is, which may include vocals, sound effects, or dialogue rather than clean instrumental beds; this was a deliberate, documented simplification for Milestone 1/dual conditioning, not an oversight, but it means the bridge’s training target is not guaranteed to be purely instrumental music.
- **No temporal modeling beyond mean-pooling.** Both frozen video encoders are mean-pooled over their 16 sampled frames with no explicit motion/dynamics fusion between the VideoMAE and CLIP streams beyond VideoMAE’s own internal attention — a clip’s fine-grained temporal structure is not otherwise modeled.
- **Excluded data.** MISMATCH clips and EmoMV’s separate `for_retrieval` subset (466 clips) are excluded from this project’s manifest by design (Section 2.2); this is a deliberate scoping decision, not an omission, but means results here are not directly comparable to numbers computed over EmoMV’s full original benchmark.
- **Version-control state.** The dual-conditioning extension’s code (`config.yaml` edits, the new `project/models/conditioning.py` module, and the corresponding changes to `train_bridge.py/generate_samples.py/08_bridge.ipynb`) exists in the working tree at the time of this report but has not been committed to version control; the only committed history is the initial pipeline commit and a second commit that adds solely the planning PDF/TeX source (`text_conditioning_bridge_plan`), with no code change.

9 Conclusion

This project builds a full frozen-backbone-plus-small-adapter pipeline from raw EmoMV video through a 5-class mood label to conditioned MusicGen output, without ever fine-tuning either the video/text mood classifiers’ large pretrained backbones or MusicGen itself. Stage A establishes an 80.1%-accuracy video mood classifier (vs. a 50.6%-accuracy SlowFast baseline); Stage B aligns video and text into a shared space reaching 81.0% video→text accuracy overall; Stage C demonstrates, first with a single-source bridge and then with a dual-conditioning extension that concatenates real T5 text tokens alongside the mood signal, that MusicGen can be steered from this shared space with zero gradient updates to the generator itself. The clearest next steps — an automatic Stage C evaluation metric, richer (non-template) text for the dual-conditioning video-anchored regime, and any of the proposed Milestone 2/3 MusicGen fine-tuning work — remain open, as documented in Section 7.